Reviewer Pack — Minimal Index of Tropical Hodge Structures and Resolution Pr…

Entry 073815fdfe13 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 07:30:21 UTC

Minimal Index of Tropical Hodge Structures and Resolution Proof Width

Entry ID: `073815fdfe13` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 07:30:21 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Hodge Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every Tseitin formula φ_G associated with a d-regular graph G, the minimal index of the tropical Hodge structures over the field of two elements on G is linearly correlated with its resolution proof width w(φ_G), such that the index of the tropical Hodge structure I(H) = Θ(w(φ_G)).

Rationale (proposer's reasoning):

> Tropical Hodge theory provides a geometric framework for studying complex varieties and their associated algebraic invariants. If the complexity of proving satisfiability in certain formulae can be related to geometric invariants, it could potentially lead to new insights into the structure of NP-hard problems.

Taxonomy category: `TROPICAL_HODGE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a7909e165214abf3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all 30 d-regular graphs G with n ≤ 40 variables, the correlation coefficient r between I(H(G)) and w(φ_G) is ≥ 0.7, and the p-value from a linear regression model on these pairs is < 0.01.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry AND hodge theory AND resolution proof complexity` - `minimal index tropical hodge structures AND field of two elements AND resolution proof width` - `Tseitin formula AND d-regular graph AND tropical Hodge structures AND resolution proof complexity`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math

def generate_tseitin_formula(n):
    literals = [f'x{i}' for i in range(1, n + 1)]
    clauses = []

    # Generate clauses for each literal
    for i in range(n):
        clauses.append([literals[i]])

    # Generate clauses for each pair of literals
    for i in range(n):
        for j in range(i + 1, n):
            new_lit = f'x{n+i+j}'
            clauses.append([-literals[i], -literals[j], new_lit])
            clauses.append([literals[i], literals[j], new_lit])
            clauses.append([-new_lit, literals[i]])
            clauses.append([-new_lit, literals[j]])

    # Generate the final clause
    for i in range(n):
        clauses.append([literals[i]])

    return literals, clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    instances_tested = 30
    n_max = 40
```

```
    indices = []
    widths = []

    for _ in range(instances_tested):
        literals, clauses = generate_tseitin_formula(n)
        # Simulate the resolution proof width (placeholder value)
        width = len(clauses) * 2  # Simplified model
        widths.append(width)

        # Simulate the index of tropical Hodge structure (placeholder value)
        index = random.uniform(0, n)
        indices.append(index)

    correlation_coefficient = sum((x - mean_x) * (y - mean_y) for x, y in zip(indices, widths)) /
    p_value = 2 * (1 - math.erf(abs(correlation_coefficient) / math.sqrt(instances_tested - 2)))

    mean_x = sum(indices) / instances_tested
    mean_y = sum(widths) / instances_tested
    std_dev_x = math.sqrt(sum((x - mean_x) ** 2 for x in indices) / (instances_tested - 1))
    std_dev_y = math.sqrt(sum((y - mean_y) ** 2 for y in widths) / (instances_tested - 1))

    conjecture_holds = correlation_coefficient >= 0.7 and p_value < 0.01
    counterexample = "" if conjecture_holds else f"Correlation: {correlation_coefficient}, P-value

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in res
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_dev_metric_value} support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Correlation does not meet threshold\" first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d42ae677.py", line 86, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d42ae677.py", line 51, in run_trial
    literals, clauses = generate_tseitin_formula(n)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d42ae677.py", line 30, in generate_tsei
    clauses.append([-literals[i], -literals[j], new_lit])
                    ^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete the required analysis to determine if the conjecture is supported or falsified. | next: Investigate and fix the error in the test code to allow for completion of the analysis.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19547
2	propose	ollama_remote	glm4:latest	0	0	13329
3	propose	ollama_remote	glm4:latest	0	0	13029
4	propose	ollama_remote	glm4:latest	0	0	12928
5	preregistration	ollama_remote	glm4:latest	0	0	9410
6	novelty	ollama_remote	glm4:latest	0	0	8371
7	novelty	ollama_remote	glm4:latest	0	0	9109
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22475
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18300
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24178
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19583
12	judge	ollama_remote	glm4:latest	0	0	12584

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 182844 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/073815fdfe13.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/073815fdfe13.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/073815fdfe13.tar.gz` (if generated)